

Knowledge Transfer report

Table of Contents

1. Purpose

- 1.1. Scope
- 1.2. Terminology
- 1.3. References

2. Key Personnel

- 2.1. Business Users
- 2.2. Subject Matter Experts (SMEs)
- 2.3. Developers

3. Technical Knowledge

- 3.1. Core Languages
- 3.2. Dependent Languages
- 3.3. Reporting Tools
- 3.4. Databases
- 3.5. Operating Systems

4. Operational Knowledge

5. Application Knowledge

- 5.1. Application Level Function

6. Application Environment

- 6.1. Flows/Diagrams
- 6.2. Server-Side Application Components
- 6.3. Client-Side Application Components
- 6.4. Users-Network Environment

1. Purpose

1.1. Scope

The scope of this document covers the mechanisms and operational knowledge of the Cable Installation Robot for Contested Environments Software (CIRCESoft). Due to logistical and time constraints, the CIRCESoft capstone team was unable to connect CIRCESoft to CIRCEBot. This document focuses on a connection to a simulation in lieu of actual connection to CIRCEBot.

1.2. Terminology

- A* Pathfinding Algorithm: Component integrated into CIRCESoft to find the shortest path to each destination while avoiding obstacles.

2. Key Personnel

2.1. Business Users

US Army Combat Capabilities Development Command (DEVCOM)
Tennessee Technological University

2.2. Subject Matter Experts (SMEs)

The following people are subject matter experts in their respective topics:

Celia Hough – Client communication and project leadership

JP Ognibene – Robot Sim and A* pathing algorithm.

Graham Wheeler, Harrison Simpson, and Ryan Naleway – JavaScript development

Gabriel Adams – Backend engineer

2.3. Developers

Detail the responsibilities and technical knowledge required for the developers involved in the knowledge transfer.

Celia Hough – Was team scrum master and took the most responsibility in handling our team-client communications.

JP Ognibene – Developed logical interactions between the Circebot simulation and Circesoft.

Graham Wheeler, Harrison Simpson, and Ryan Naleway – JavaScript development

Gabriel Adams – Backend engineer

3 – TECHNICAL KNOWLEDGE

3.1. Core Languages

Python 3 Used for backend logic, including the API server, pathfinding system, simulation driver, and supporting utility scripts.

JavaScript (JSX) Used for the frontend application built with React. Handles user interaction, data displays, and WebSocket communication.

CSS Used for the visual layout, theming, and accessibility styling of the frontend interface.

3.2. Frameworks and Libraries

Backend (Python):

- FastAPI is the main framework supporting the asynchronous REST API and the WebSocket communication endpoints.
- Uvicorn and Gunicorn are the ASGI servers used to run the FastAPI application efficiently in production.
- Asyncio is used for concurrent operations such as WebSocket broadcasting and continuous monitoring tasks.
- aiohttp is used by the A* pathfinding worker to retrieve environment data and return computed paths back to the server.
- python-dotenv is used to load configuration values from a .env file.

Frontend (JavaScript / React):

- React is the primary framework used to build the single-page frontend interface.
- Vite is the development and build tool used to serve, optimize, and bundle the React project.

3.3. Reporting and Status Updates

The system uses an internal real-time reporting mechanism rather than an external analytics platform.

- WebSockets are used for fast status updates. The server broadcasts robot status, obstacle changes, A* results, and other live data to connected clients roughly every 25 milliseconds.
- JSON files are used as shared state files. The simulation or robot writes to files such as `current_values.json`, and the server reads and broadcasts their contents.

- A* status updates are sent by the pathfinding worker through a dedicated POST endpoint and then forwarded to clients through a separate WebSocket channel.

3.4. Data Storage

The system does not use a traditional SQL or NoSQL database.

All persistent data is stored in JSON flat files inside a designated directory called GRID_DIR (default path: ./data). These files include waypoints.json, obstacles.json, current_values.json, and grid_path.json.

This approach keeps the architecture simple but requires a shared persistent storage volume when deployed in a clustered or containerized environment.

3.5. Operating Systems

The OS that is used based on the given project requirements is Ubuntu 22.04.

4. Operational Knowledge

To operate CIRCESoft, the user must first deploy the Docker container on a device and connect the device to CIRCEBot. From there, the user can specify paths and obstructions. While CIRCEBot is (as of December 2025's engineering specifications) equipped with LiDAR for automated collision avoidance, it is heavily recommended that the operator manually plot any obstacles that may interfere with CIRCEBot's path in order to ensure the path calculations are relevant to the maximum C2 cable length of 300 feet.

CIRCESoft has built-in protections to prevent a user from creating a path longer than 300 feet. In its current state, a message will be sent to the message window if a user tries to make a waypoint that exceeds the maximum operating distance of CIRCEBot. Users are not able to move waypoints to areas that exceed this distance as well.

Once a path has been plotted, it can be sent to the A* pathfinding algorithm by pressing the "Calculate New Path" button located on the left side of the screen. A* will return the path to the tactical map with new gridpoints added to help CIRCEBot reach its destination. If the shortest path is greater than 300 feet including the added distance needed to maneuver around obstacles, the message window will show an error and the tactical map's path will not be affected.

CIRCESoft is capable of "backtracking" over path that it has already progressed over in order to get to its destination, i.e. coordinates can appear on the list twice. This is useful for edge cases in which a waypoint is marked in an area that is surrounded by obstacles and has only one way in and out.

Telemetry data at the bottom of the operator panel shows important metrics such as battery life and remaining cable length. It is important to note that once CIRCEBot is set along a path, CIRCEBot will continue the path **even if disconnected from CIRCESoft**. This means that in the event of disconnection, CIRCEBot can still traverse the path to its destination, just without sending telemetry data back to CIRCESoft.

5. Application Knowledge

5.1. Application Level Function

Explain the primary functions and features of the application. (the docker container)

6. Application Environment

6.1. Flows/Diagrams

Describe the flow of data or processes within the application, potentially using diagrams or visual aids.

6.2. Server-Side Application Components

The server-side application for the CirceSoft Control System is built using a Python-based microservice architecture. It is divided into two separate processes: the FastAPI Control Server, which functions as the central communication hub, and the A* Pathfinding Worker, which performs all computational pathfinding tasks.

FastAPI Control Server (Core Backend)

The FastAPI Control Server handles all REST API requests, WebSocket connections, and manages shared application state stored in JSON files.

app.py

This is the main entry point for the backend application. It initializes the FastAPI server, sets the application title to CirceSoft Control Server, applies the CORS middleware using configuration values from config.py, and loads the main router contained in routes.py.

routes.py

This file defines all REST API endpoints such as GET and PUT operations for waypoints, obstacles, and current robot values. It also manages all WebSocket connections through a ConnectionManager class. The server handles the core broadcasting logic, pushing

updated data, including robot status, obstacle changes, and path updates, approximately every 25 milliseconds.

config.py

This file loads all environment variables using python-dotenv and defines key settings such as the host address (0.0.0.0), the default server port (8765), the WebSocket path (/ws), and file paths for the shared JSON data directory (./data).

utils.py

Contains helper functions such as reading and writing JSON files, parsing grid coordinates, and motion detection logic. Key functions include `calculate_distance` for cable tracking and `is_bot_moving_ll` for processing robot status.

models.py

Defines persistent global state variables, including `CABLE_REMAINING` (initialized to 50.0m) and `LAST_POSITION_ECI`. It also includes mock Protobuf structures that simulate real robot messaging formats for future hardware integration.

A* Pathfinding Worker (`grid_astar.py`)

This component runs independently of the FastAPI server to avoid blocking the main application during heavy computation.

Primary Function

Computes the optimal path between user-defined waypoints while respecting the maximum allowed cable length constraint of 300.0 feet.

Grid System

The A* pathfinder operates on a fixed 31 by 48 grid. The coordinate system uses 0,0 as the bottom-left corner of the grid.

Asynchronous Watch Loop

The script uses an asynchronous watch loop that continuously monitors the FastAPI server for changes to waypoints and obstacles. It recalculates the path only when needed and enforces a two-second cooldown timer to prevent excessive recalculation when inputs change rapidly.

The worker uses the `aiohttp` client to retrieve input data from the server via GET requests (for waypoints and obstacles) and to send the computed path back to the server using a PUT request to the `grid/path` endpoint. Status messages or error conditions are reported to the server using the `send-astar-message` POST endpoint, which is then broadcast to clients.

Shared Data Storage (Data Flow Mediator)

The file system acts as the shared data layer between the Control Server, the A* Worker, and the Simulation Driver. All application state is stored in JSON files inside the designated data directory (GRID_DIR).

The waypoints.json file is written by API clients through routes.py and read by the A* Worker. It contains the waypoint list used as input for pathfinding. The obstacles.json file is also written by API clients and contains the grid obstacles used during path calculation. The grid_path.json file is written by the A* Worker and read by both the FastAPI server and any connected API clients. It contains the computed A* path. Finally, the current_values.json file is written by the simulation driver script and read by the FastAPI server and connected clients, containing real-time robot position and status data. This file-based storage system ensures clean separation between components and avoids dependencies on a traditional database.

6.3. Client-Side Application Components

The CirceSoft frontend is built using the following core technologies:

React – A JavaScript library for building component-based user interfaces. React manages the application state and renders UI updates efficiently when data changes.

Vite – A modern frontend build tool that provides a fast development server with hot module replacement and optimized production builds.

WebSocket API – The native browser WebSocket API is used to maintain persistent, real-time connections with the backend for receiving telemetry updates, path changes, and system notifications.

CSS – Custom stylesheets with a consistent dark theme using green accents and Courier New monospace fonts for a professional, terminal-like aesthetic.

Core client-side components:

App.jsx – The root component that manages global state, initializes the WebSocket context, and defines the overall layout structure of the application.

MapView.jsx – The interactive grid component where users draw paths, place obstacles, and visualize the bot's real-time progress along its route.

PathControls.jsx – A control panel providing mode toggling between path and obstacle editing, as well as import/export functions and path management options.

MessageWindow.jsx – A scrollable log panel that displays system messages, warnings, errors, and status updates from both the frontend and backend.

Progress.jsx – A visual progress bar that compares distance traveled against total path length to show completion percentage.

StartStopButton.jsx – Controls for starting and stopping bot execution, with visual feedback indicating current running state.

CurrentValuesDisplay.jsx – A real-time telemetry panel displaying live data including position, heading, battery percentage, and cable remaining.

Websocket.jsx – A context provider component that manages the WebSocket connection and distributes live telemetry data to all child components via React Context.

The frontend communicates with the backend through two channels: REST API calls (using fetch) for sending user commands and retrieving static data, and WebSocket connections for receiving real-time updates. The WebSocketProvider component wraps the application and exposes live telemetry data to child components via React Context.

6.4. Users-Network Environment

Users

Primary Operators – Military warfighters responsible for controlling CirceBot in contested environments. These users interact with the CirceSoft interface to define waypoints, place obstacles, monitor bot progress, and issue start/stop commands. Operators are expected to have basic familiarity with the interface but do not require technical or programming expertise.

Developers – Software or computer engineering personnel responsible for maintaining, updating, or extending CirceSoft functionality. Developers work directly with the React frontend, FastAPI backend, A* algorithm, and simulation components.

Network Environment

Air-Gapped Operation – CirceSoft is designed to operate in an isolated, air-gapped environment with no connection to external networks or the internet. This architecture minimizes the attack surface and ensures high availability in mission-critical scenarios.

Localhost Communication – All communication between the frontend, backend, and A* pathfinding algorithm occurs over localhost (127.0.0.1). The backend runs on port 8765 by default, and the frontend development server typically runs on port 5173 or 3000.

Physical Tether Connection – CirceSoft communicates with CirceBot via a hardwired Ethernet cable. This tethered connection provides reliable, low-latency communication without the vulnerabilities associated with wireless transmission.

No External Dependencies – The system does not rely on cloud services, external APIs, or online resources. All computation and data storage occur locally on the operator device, ensuring the system remains functional in environments without network connectivity.

Security Considerations

Physical access to the operator device serves as the primary access control mechanism. Because the system operates in an air-gapped environment within secured military installations, network-level authentication is not implemented. The Ethernet tether represents the primary security concern, as it could potentially be spliced by an adversary in a contested environment.

JP Ognibene

Gabriel Adams

Ryan Naleway

Celia Hough

Elias Brady

Joze Lindquist